

Tool Designed for translation from *Mediator* to Z3 Python and verification of its validity

Siyi Song and Zichen Tang

Peking University, No.5 Yiheyuan Road, Haidian District, Beijing, 100871, P.R.China

Abstract. As a specific language targeted at modeling and verification, *Mediator* is developed to describe properties in normalized files, and to realize its functions through a set of tools. In this article, we focus on the design of a simple translation tool from *Mediator* to Z3 Python encodings suitable for SMT-based verification. With a basic understanding of the language *Mediator* itself, we try to convert it into Z3 Python by parsing it into AST and dealing with type mapping, function generation, variable declaration, transitions etc. to use the mature solver Z3 to verify the validity of given models. Two specific examples are included in this article as a preliminary attempt. . . .

Keywords: Mediator, Z3, language converter, verification

1 Introduction

With the structure of hardware systems getting more and more complicated, formal verification is needed to ensure validity. Unlike conventional testbench method which only covers limited situations and may miss potential mistakes, domain-specific modeling languages (DSL) such as *Mediator* enable compact, expressive descriptions of automata and testbench scenarios.

Mediator is often used to describe hardware structure, signal interaction and testbench environment, while Z3 is a widely applied SMT solver for hardware property verification. Further introduction of *Mediator* and Z3 will be included in the next section.

Despite the fact that *Mediator* is convenient for modeling, there are currently limited support to translate *Mediator* to other languages so that powerful SMT solvers such as Z3 can be used to perform automated verification using , making it hard to directly apply *Mediator* models to formal solution.

In this article, we make an attempt to build a translation tool from *Mediator* to Z3 Python, with two examples as tests of validity. A *Mediator* 'Automaton' or 'System' is taken, and flattened, and its structure is normalized. Then a Z3 script which encodes a bounded unrolling of system behavior is emitted, enabling bounded model checking and other SMT-based verification techniques.

⁰ Siyi Song is responsible for writing this article and provides some ideas for the realization of the project, while Zichen Tang mainly contributes to the design of the programming part. Both authors contributed equally to this project and are designated co-first authors.

This document is structured as follows. Section 2 covers background on *Mediator* and Z3. Section 3 reviews related work. Section 4 formalizes the translation and explains implementation details. Section 5 presents the Testbench and Medical System case study and verification setup. Sections 6 discusses results, limitations, and future work.

2 Some Background Information

Now that we are trying to convert *Mediator* to Z3 Python, we must have an overall understanding of the language and the solver. Here is the basic introduction which is useful in our work.

2.1 The language *Mediator*

Mediator is a DSL for modeling reactive systems using entities like automata and systems, and is often used to specify interactions among components.

Types of terms *Primitive types* include common types that compose a powerful type system: integers(int), bounded integers(int l $\cdots$ r), real numbers(real), boolean values(bool), characters(char) and finite enumerations(enum *item*₁, $\cdots$, *item*_n).

Composite types are more complex, and are, at most times, combination of *Primitive types*. Examples include Tuple, Union(a combination of different types), Array and List(static and dynamic list of variables), map, struct, etc.

Parameter types can be any concrete type, and include abstract type and interface type.

Remark Subtyping rules indicate that certain order relations exist between different types, e.g. bounded integers can be assigned to the int type. This is an important aspect in verification.

For the purpose of translation, we assume operational semantics where at each global step one process executes a single transition (interleaving semantics).

Functions, Automaton and System *Functions* in *Mediator* can be simplified into mathematical functions in that they involve no external variables. *Automaton* plays an important role in the work of formal verification. It contains local variables and typed ports.

What's unique about it is *transitions*, which are statements describing the update of variables being operated if certain conditions are satisfied. Such activating conditions are called guard, hence we denote the transition with guard $\rightarrow$ statements. *Statement* in *automata* are divided into two categories. Firstly, we have assignment statements that update new values. We also have synchronizing statement when joining multiple automata. *System* is a collection of smaller components like *automata* to make it easier to use.

Something more about automaton Here we have a simple mathematical description of automaton.

Guards: $g : Conf(A) \rightarrow Bool$ Here $Conf(A)$ is the set of all configurations, and a configuration of an automaton $A = \langle Ports, Vars, Trans_G \rangle$ is defined as a tuple (e, P) where $e \in EV(Vars \cup Adj(A))$ is an evaluation on both local variables and port variables, $P \subseteq Ports$ logs the ports which are synchronized in the latest transitions.

Assignment Statements: $s_a : Conf(A) \rightarrow Conf(A)$

Standardize the expressions Since quite different statement combinations can actually share the same meanings, we need to introduce a canonical form of transitions and automata.

Converting a hierarchical system to an automaton

1. Turn all entities into canonical automata
2. Rebuild the flattened automaton
3. Put the transitions together.

The execution of an automaton can be regarded as a discrete time sequence, where in each step we:

1. Evaluate the guard of all transitions, choose one transition whose guard is satisfied (if there is more than one, then we choose at random according to the semantics or let the scheduler decide)
2. Execute the statements of the transition and update variables
3. Move on to the next step

Remark We need to be aware of external transitions since they must synchronize with other transitions in other entities. However, from the aspect of the labeled transition system, the only difference between an internal transition and an external transition is that the latter may contain more than one assignments.[2]

2.2 Satisfiability Modulo Theories Solver Z3

Satisfiability Modulo Theories, or SMT, are basically a decision problem for logical formulae with respect to background theories like arithmetic, bit-vectors, arrays and uninterpreted functions. It is, in fact, an extension of SAT. Designed by Microsoft Research, Z3 is a mature, high-performance, state-of-the-art and widely used SMT Solver with specialized algorithms that support various theories and languages like linear arithmetic, uninterpreted functions, arrays, bitvectors, etc. Here we use the Python API for execution and result parsing, which provides convenient constructors for creating Int, Real, Bool, building formulae and invoking solver queries. Under circumstances of bounded model checking, we often unroll system transitions for a certain finite bound. That is, listing out all the possible status at step k of the system, encoding the status and transition into SMT constraints and using Z3 to find counterexample (as long as status

at one point violates the safety property), which implies the invalidity of the system.[3][4][7]

Key functions of Z3 includes:

1. Variable Declaration: `<type>('variable_name')`
2. Logic: `And/Or/Not(...)`
3. Condition: `If(cond, then, else)`
4. Solver control: `Solver(),s.add(),s.check(),s.model()`
5. Recall: `s.push(),s.pop()`

Common steps include:

1. Declaration of all variables
2. Add initial conditions
3. Add `Or(And(guard,updates))` of transition to each step
4. Add `(Or(Not( $P_0$ ),...,Not( $P_k$ ))) $\rightarrow$ s.check()`

If, however, further examination is required, push/pop can be applied to test different k or constraints using the same solver.

2.3 ANTLR and Parse Tree

ANTLR ANTLR, or Another Tool for Language Recognition, is a popular parser-generator. A grammar file in the format of .g4 is used to describe the semantics of the language, and ANTLR will generate a Lexer(which breaks the code into tokens) and a Parser(which assembles the tokens into a parse tree).

In this project, ANTLR is responsible for converting the source code in the format of .med to a parse tree before turning it into more friendly AST.

Parse Tree Parse tree is a tree-shaped structure generated directly from grammar, with each node a grammar rule or token. It contains the complete grammar information and is usually much longer than AST.

Parse tree is the mirror of the semantics that keeps all the details, while AST is a simplified and abstract structure that only contains the semantically important nodes.

Usually the following steps are included:

1. Read the source code .med
2. Create Lexer and Parser using ANTLR, and generate parse tree
3. Traverse the parse tree and map the grammar nodes to AST items
4. AST can be directly used by Generator/Simulator after semantic analysis.

In our work, we directly traverse AST.

3 Related Work

3.1 Hardware Verification using SMT Solvers

Contemporary hardware verification increasingly uses SMT solvers as tools for bounded and symbolic checks. Early and foundational work on bounded model checking (BMC) demonstrated how unrolling transition relations and solving the satisfiability problem can find bugs efficiently.[8] Toolchains such as the Yosys project [9] and its SMT based model checkers translate Verilog/RTL into SMT formula solvers like Z3 to carry out property check and bounded proofs. Such toolchains are mature and effective for standard hardware description languages (HDLs), but they are intended for Verilog/VHDL input, which is low-level semantics in the translation to SMT.

3.2 Domain-Specific Languages for Testbench or Hardware Modeling

Domain-specific languages and frameworks are widely applied in hardware verification to express testbenches, stimuli, and behavioral scenarios at higher abstraction levels than RTL. Industrial practice relies heavily on SystemVerilog together with the Universal Verification Methodology (UVM) for constructing rich, reusable testbench components and constrained random stimulus. SystemVerilog/UVM provides extensive libraries and a methodology tuned for large-scale verification [11]. Complementary commercial and legacy DSLs such as Specman/e and assertion languages like PSL (Property Specification Language) are also used to express monitors, properties and checkers in verification flows [12]. At a slightly different abstraction level, SystemC and Transaction-Level Modeling (TLM) support cycle-approximate or untimed modeling of communication and are commonly used for early-stage verification and architectural exploration [13].

In the formal methods research community, specialized modeling formalisms such as UPPAAL for timed automata and Promela/SPIN for asynchronous protocol checking provide domain-focused languages and toolchains tailored to particular classes of verification problems [14]. These DSLs emphasize behavioral modeling and property specification rather than gate-level implementation details.

Compared to these established languages, *Mediator* fits the category of a high-level modeling DSL that focuses on automata, guards, transitions and properties in a readable, structured way. Unlike Verilog/VHDL which are geared towards cycle-accurate hardware semantics, or SystemVerilog/UVM which targets industrial testbench engineering, *Mediator* is more abstract and centered on behavior: it is intended to describe component interactions, test scenarios and properties at a level that is closer to system behavior than to synthesizable RTL. This difference motivates a dedicated translation and encoding strategy: instead of preserving cycle-accurate bit-level semantics (as many HDL→SMT pipelines

do), a *Mediator*→SMT approach can exploit the language’s higher-level structure (for example, by flattening structs to fields, encoding enums as named integer constants, and mapping properties directly to state-formula checks) and focus on verifying behavioral correctness and testbench properties.

3.3 Program/Model Translation to SMT

There have been various projects on translation of programming languages and models into SMT (or SAT) to enable automated verification. In software verification, Bounded Model Checking (BMC) and SMT-based symbolic checking have been effectively applied to C and embedded C programs by translating program control-flow and data manipulations into quantifier-free formulae that are decided by modern solvers [15]. Intermediate verification languages such as Boogie separate front ends from back ends, serving as an Intermediate Representation that is well suited for subsequent translation to SMT and modular verification architectures [16].

In model-based verification, numerous projects translate state-based models (statecharts, UML state machines, timed automata) into target logics or tools (SMV, Promela, SMT encodings) to check reachability and temporal properties; the general recipe is to construct an intermediate representation of states and transitions, unroll them for a bounded depth when performing BMC, and hand final quantifier-free formulae to an SMT/SAT engine for decision and counterexample generation [17]. For hardware, toolchains such as Yosys[9] with its smtbmc/SymbiYosys front ends translate Verilog/RTL nets into SMT and use solvers to perform bounded checks. These systems illustrate how front end semantics (e.g., bit-vector operations, synthesis transforms) shape SMT encodings and solver performance.

3.4 Bounded Model Checking (BMC)

Here we introduce the Bounded Model Checking in detail, as it is used in our program.

Bounded Model Checking, or in short, BMC, is a kind of verification method on an automatic model based on SAT/SMT. Usually it checks whether the system violates some certain properties (usually safety properties) by searching on a given number of steps.

Different from traditional searching method which uses BFS or DFS after building a graph, BMC flattens the "reachability of a bad state" problem to a boolean or logical formula under steps of a fixed length k , and then gives it to SAT/SMT solver. If the conditions in the formula can be satisfied, the solver will return a counterexample with a length no more than k . [18]

Therefore, we will need the starting point $I(s)$, transition $T(s,s')$ and safety property $P(s_0, \dots, s_k)$ (the bad state would then be $\neg P$). The formula we construct to test the step length k will be:

$$I(s_0) \wedge T(s_0, s_1) \wedge T(s_1, s_2) \wedge \dots \wedge T(s_{k-1}, s_k) \wedge (\neg P(s_0, s_1, \dots, s_k)) \quad (1)$$

If the formula can be satisfied, then there exists a path with a length no more than k but violates P , and the solver will return a concrete sequence. Otherwise, there are no counterexamples. A **sat** result yields a counterexample trace, while **unsat** indicates no counterexample up to bound k . Note that the model itself should be reasonable, or there can be contradictions at some certain steps.

It is worth noting that BMC cannot directly prove the security of infinite steps, and counterexamples too long will be stressful for the solver.

Our work follows the same high-level pattern — parse source models, form a compact intermediate representation, and emit bounded SMT encodings, but it targets the *Mediator* language specifically. *Mediator*’s AST (with structs, enums, unions and named properties) suggests concrete, pragmatic encoding choices (flattening composite types into atomic fields, representing enums as integer constants, using time-indexed variables v_t for unrolling, and encoding invariants as disjunctions of negated state formulae over time). These choices differ from those in general purpose software front ends and from HDL $\rightarrow$ SMT flows which emphasize bit-precise, cycle-accurate semantics. By focusing on *Mediator*’s semantics and verification use cases, our generator aims at a practical and readable translation that helps verification engineers and researchers use Z3 directly on higher-level behavioral models.

4 Design of the *Mediator* to Z3 Translation Tool

4.1 Overview of the Translation Process

The translation process from *Mediator* models to Z3 constraints involves four main stages:

1. **Input Parsing:** the *Mediator* source file is parsed into AST.
2. **Semantic Processing:** the system performs type checking, scope analysis, and flattens the hierarchical automaton structure into a global transition system.
3. **Constraint Encoding:** the flattened model is encoded into SMT constraints using a bounded unrolling strategy.
4. **Verification:** the generated Python script invoking the Z3 solver is executed to check for counterexamples.

4.2 Design principles

1. **Semantics Preservation:** The encoding must faithfully preserve the semantics of the *Mediator* subset targeted. Specifically, any execution trace satisfying the Z3 constraints must correspond to a valid execution trace in the original *Mediator* model.
2. **Modularity:** The translation supports pre-processing to enable compositional reasoning, allowing components to be verified in isolation or as a composed system.
3. **Simplicity and Performance:** We select encodings that are both compact for generation and efficient for the Z3 solver to handle, avoiding unnecessary possible quantification.

4.3 Realization of the Tool: Z3 Generator for *Mediator*

We implemented a Z3 generator that emits a Python script performing Bounded Model Checking (BMC). The core approach involves **flattening** the complex parallel system into a single global automaton and encoding its execution traces into SMT constraints over a finite time horizon k .

Type mapping *Mediator* is equipped with a rich type system. In this project, we map these types to the fundamental sorts supported by Z3: `Int`, `Bool`, and `Real`.

- **Primitive Types:** *Mediator* `int` and `double` are mapped to Z3 `Int` and `Real` respectively.
- **Enumerations:** Enum types are encoded as integers, with constraints ensuring that they remain within valid ranges, by building a group of global variables to record the relationship between the enum constant and int values.
- **Complex Types:** For structured types (structs), we adopt a **flattening strategy**. A struct variable p with fields x and y is flattened into distinct variables $p.x$ and $p.y$. This avoids the overhead of Z3’s algebraic data types and simplifies constraint generation.

To handle arbitrarily nested data structures (e.g., a struct containing arrays of structs), we employ a recursive flattening algorithm (Algorithm 1). This procedure traverses the type hierarchy depth-first. For a variable v of type T , it generates a set of flat variables by concatenating field names or indices with underscores. This ensures that a complex state space is deterministically mapped to a flat vector of SMT primitives.

Algorithm 1 Recursive Variable Flattening

```

1: procedure FLATTEN(String prefix, Type  $T$ , Map  $M$ )
2:   if  $T$  is Primitive Type (Int, Bool, Real) then
3:      $M.add(prefix, T)$ 
4:   else if  $T$  is StructType then
5:     for each field  $f \in T.fields$  do
6:        $newPrefix \leftarrow prefix + \_ + f.name$ 
7:       FLATTEN( $newPrefix$ ,  $f.type$ ,  $M$ )
8:     end for
9:   else if  $T$  is ArrayType then
10:    for  $i \leftarrow 0$  to  $T.size - 1$  do
11:       $newPrefix \leftarrow prefix + \_ + i$ 
12:      FLATTEN( $newPrefix$ ,  $T.elementType$ ,  $M$ )
13:    end for
14:   end if
15: end procedure

```

Function generation Functions in *Mediator* are designed to be side-effect free, with accuracy and preciseness resembling mathematical formulae. This characteristic allows for a direct mapping. We translate *Mediator* functions into Z3 function definitions or inline expressions, ensuring that the functional logic is preserved within the constraints without needing to model a call stack.

Variable declaration Since variables in *Mediator* change over the running process, we employ a **Bounded Unrolling** strategy (often referred to as Static Single Assignment form in BMC contexts). For each variable v in the flattened model and for each time step $t \in [0, k]$, we instantiate a distinct Z3 variable v_t .

$$v \mapsto \{v_0, v_1, \dots, v_k\} \quad (2)$$

This allows us to model the state of the system at any discrete time step t using standard logic, or Z3 solver may assign random values to the initial state of these variables, leading to the appearance of fake counterexamples. For example, assume we have a component a , with an input interface p whose value type is $p_{value} : int$, and the property we hope it would satisfy is that the value p gets is always more than 2. If we normally set the initial value of int to 0, a counterexample will come into being (unsat will never be satisfied unless the model violates other properties), as long as the model itself is not contradictory, and we will always get that the result of not satisfied, which, apparently, is unreasonable.

Transition Encoding We use constraint statement to express transitions. A typical transition in *Mediator* consists of guard and statement. In an automaton that is generated by scheduling, all transitions are in a transition group, and every transition is canonical[2]. We encode the system's evolution by generating constraints that link the state at time t to the state at time $t + 1$.

- **Guards and Updates:** For each transition τ , we generate a guard constraint $G_\tau(t)$ and a set of update constraints $U_\tau(t, t + 1)$. If the guard is true, the updates are applied.
- **The Frame Problem:** A critical aspect of the encoding is handling variables that are *not* modified by a transition. We explicitly generate **Frame Conditions** to ensure that if variable x is not assigned in transition τ , it retains its value: $x_{t+1} = x_t$. That is why we maintain a list in the realization to take a record of the variables in the transitions that have been assigned values and the corresponding assignment items.

The transition relation for step t is the disjunction of all possible transitions, ensuring that at least one valid transition (or a stuttering step) occurs.

Note According to [2], canonical transition is defined to be statement after guard without sync statement —single assignment statement with the form of $a, b, c, d, \dots = t_1, t_2, t_3, t_4, \dots$ (here t_i is item/expression).

It is worth mentioning that on the one hand such way of assignment is not naturally supported. In the real parsing process, it will be interpreted as tuple

assignment, yet currently we only support single variable assignment and struct assignment. (In fact, we assign values to the flattened struct, hence assigning values to multiple variables)

On the other hand, such way of assignment almost naturally rejects the existence of the situation mentioned above, where a variable is assigned a value before it is used to assign another variable. This is its upside.

Property Verification Unlike simple invariant checking, our tool supports a subset of Linear Temporal Logic (LTL). We implement a **recursive translation function** $\mathcal{T}(\phi, t, k)$ that maps a temporal formula ϕ at time t to a Z3 boolean expression.

The translation rules are defined recursively:

- **Atomic Propositions:** $\mathcal{T}(p, t, k)$ is the direct constraint of p on state variables at time t .
- **Globally ($G\phi$):** The property must hold at all future steps up to k .

$$\mathcal{T}(G\phi, t, k) = \bigwedge_{i=t}^k \mathcal{T}(\phi, i, k)$$

- **Finally ($F\phi$):** The property must hold at some future step.

$$\mathcal{T}(F\phi, t, k) = \bigvee_{i=t}^k \mathcal{T}(\phi, i, k)$$

- **Next ($X\phi$):** The property must hold at the next step.

$$\mathcal{T}(X\phi, t, k) = \text{if } t < k \text{ then } \mathcal{T}(\phi, t+1, k) \text{ else False}$$

To verify a property P , we assert the negation of its translation: $\neg\mathcal{T}(P, 0, k)$. If the solver finds this satisfiable, the resulting model represents a counterexample trace violating P .

Type Safety and Boundary Verification Beyond temporal properties, our tool is capable of verifying type safety constraints, specifically for bounded types such as `BoundedInt`. Since Z3’s `Int` sort is unbounded, we generate additional constraints to ensure variables remain within their declared ranges throughout the execution:

$$\bigwedge_{t=0}^k (v_t \geq v_{min} \wedge v_t \leq v_{max})$$

In the current implementation, this feature is treated as an optional module (enabled by uncommenting specific lines in the generated script) to prioritize performance for standard property checking.

Future Work: Enhancing the user feedback mechanism—such as pinpointing exactly which property was violated and at which time step—remains a key direction for future improvements to make the tool more developer-friendly.

4.4 Core Algorithm Realization

The implementation logic is summarized in Algorithm 2. The algorithm iterates through time steps to build the transition relation and then applies the recursive property translation.

Algorithm 2 Mediator to Z3 Constraint Generation

```

1: procedure GENERATEZ3CONSTRAINTS(Automaton  $A$ , int  $k$ )
2:    $S \leftarrow \text{NewSolver}()$ 
3:   Define variables  $v_t$  for all  $v \in A.vars, t \in [0, k]$ 
4:   Assert  $\text{InitConstraints}(A, t = 0)$ 
5:   for  $t \leftarrow 0$  to  $k - 1$  do
6:      $\text{TransExpr} \leftarrow \text{False}$ 
7:     for each transition  $\tau \in A.transitions$  do
8:        $\text{Guard} \leftarrow \text{Translate}(A.guard_\tau, t)$ 
9:        $\text{Updates} \leftarrow \text{Translate}(A.assign_\tau, t, t + 1)$ 
10:       $\text{Frame} \leftarrow \bigwedge \{v_{t+1} == v_t \mid v \notin A.assign_\tau\}$ 
11:       $\text{TransExpr} \leftarrow \text{TransExpr} \vee (\text{Guard} \wedge \text{Updates} \wedge \text{Frame})$ 
12:    end for
13:    Assert  $\text{TransExpr}$ 
14:  end for
15:  for each property  $P \in A.properties$  do
16:     $\text{VerificationCond} \leftarrow \text{RecursiveTranslate}(P, 0, k)$ 
17:    Assert  $\neg \text{VerificationCond}$ 
18:  end for
19:  Check  $S$ 
20: end procedure

```

This algorithmic structure ensures that both the system dynamics (including the frame conditions) and the temporal properties are rigorously encoded. The use of recursive translation for properties allows for checking complex nested temporal requirements beyond simple invariants. Here we give some more details about the functions we designed in the Java code.

generatePreamble(sb) This function serves as preparation for the project importing Z3 Python and what follows is using RealVal and ToReal to convert all integers and floats to real values that can be identified by Z3 Python.

prepareVariables(autom)

- Get the local variable declaration set vars.
- Clear or reset variables used for storage or map, and set the enumeration counter to 0
- Traverse every VariableDeclaration (that may contain several identifiers id): For each id, analyze its type, deal with it accordingly, and put the results into

varFields (key= variable id, value= LinkedHashMap that stands for string→type map).

generateEnumDefinitions(sb) This function maps enumerate items to texts and puts them together with StringBuilder. By traversing the enumItemConstMap and the Map it, we convert the name of the constant and the present serial number to a line of texts and append it to sb.

generateVariableDeclarations(sb, k) This function is used to generate atomic variable declaration texts in Z3 Python for each field of each variable at each time step according to varFields that have been collected and add them to sb.

generateInitConstraints(sb, autom) This function generates and combines initial constraint texts in Z3 Python, sets initial values for local variables of automata and add these constraints into Solver.

generateTransitionConstraints(sb, autom, k) This function generates all possible transition Z3 constraints at each time step t (from 0 to $k-1$): for every time step we combine all the transitions with $\text{Or}(\dots)$, each transition consisting of its guard and a next status equality of the variable (to be updated all stay unchanged).

generatePropertyVerification(sb, autom, k) together with the function **String propertyToCheckExpr(StateFormulae Formula, int k)** and **generateTypeSafetyVerification(StringBuilder sb, int k)** We have explained the design principles and role of the functions in 4.3 and 4.3.

generateFooter(sb) This function is used to add the ending script for Z3 Python to StringBuilder to check the satisfiability and print counterexamples when satisfied.

5 Case Studies: Verification of *Mediator* Models

In this section we analyze two *Mediator* model cases using Z3 Solver to test the related properties.

5.1 Testbench Model

Model Description The testbench model encodes a small coordination scenario with:

- An output interface process (**s_OUT**) exposing **value**, **reqRead**, and **reqWrite** signals.

- An input interface process (`m_IN`) exposing `value`, `reqRead`, and `reqWrite`.
- A smoother module maintaining an integer counter `_smoother_0_count` and average `_smoother_0_avg`.
- Control locations encoded as integer variables `s_p.t`.

The translator unrolled the model within the bound of 40 ($t=0,1,\dots,40$). The generated Python file declares each variable for every step; initial conditions are asserted at step 0. For each timestep, the transition relation is a disjunction of six alternative guarded actions, each representing a potential *Mediator* transition. A fragment of the generated time-indexed declarations (for clarity) is given below.

Listing 1.1. Excerpt of generated declarations (`testbench_z3-check.py`)

```
s_p_0 = Int('s_p_0')
s_OUT_value_0 = Real('s_OUT_value_0')
s_OUT_reqRead_0 = Bool('s_OUT_reqRead_0')
...
_smoother_0_count_0 = Int('_smoother_0_count_0')
...
s.add(s_p_0 == 0)
s.add(s_OUT_value_0 == 0.0)
s.add(s_OUT_reqRead_0 == False)
...
# step 0 transition (Or of clauses)
s.add(Or(
And( And(Not(False), Not(s_OUT_reqRead_0 == s_OUT
_reqWrite_0)),
s_OUT_reqWrite_1 == s_OUT_reqRead_0,
s_p_1 == s_p_0, ...),
And( And(Not(False), m_IN_reqRead_0 == False),
m_IN_reqRead_1 == True, s_p_1 == s_p_0, ...),
...
))
```

Property under Verification The property (called `m.safe` in the repository) asserts that the input value remains below 2:

$$\forall t \in \{0, \dots, 40\}. m_IN_value_t < 2$$

The negation is encoded as:

$$\bigvee_{t=0}^{40} \neg(m_IN_value_t < 2)$$

and passed to Z3. If Z3 returns `sat`, the model gives a violating timestep and concrete values for the variables.

Listing 1.2. Excerpt of the output (`testbench_2.py`)

```

Step 20:
    s_p:5
    s_OUT_value:4
    s_OUT_reqRead:False
    s_OUT_reqWrite:False
    m_IN_value:0
    m_IN_reqRead:True
    m_IN_reqWrite:True
    _smoother_0_count:5
    _smoother_0_avg:10
-----
Step 21:
    s_p:5
    s_OUT_value:4
    s_OUT_reqRead:False
    s_OUT_reqWrite:False
    m_IN_value:2
    m_IN_reqRead:False
    m_IN_reqWrite:False
    _smoother_0_count:0
    _smoother_0_avg:0
-----

```

5.2 Medical System

Model Description This file is generated by our tool and flattened chronologically. It models the interaction among the output interface of the sensor, the input interface of the monitor and a safety controller. Each interface has three signals at each time step t : `value`, `reqRead` and `reqWrite`.

For each time step $t \rightarrow t+1$, the translator generates an Or statement to describe the possible branches of guarded action. Each branch is constrained by several guards. Typical branches include:

- Empty/Maintaining branch: When nothing special happens, some signals are duplicated or switched at the next step.
- Monitor launching request to read branch: `monitor_IN_reqRead=True`
- Update sensor_OUT_reqRead according to `has_data` in `SafetyController`
- Control of monitor_IN_reqWrite
- Output/Collection branch (key branch): when `sensor_OUT_reqWrite.t == True` and satisfies a series of handshake conditions, `SafetyController` captures the current bpm of the sensor
- Monitor consumption behavior: when `monitor_IN_reqWrite` is triggered and the handshake conditions are satisfied, the monitor reads `buffered_status` and clears `has_data`.

The translator declares variables and add constraints one by one for each time step.

Listing 1.3. Excerpt of generated declarations (`MedicalSystem.z3-check.py`)

```
sensor_current_bpm_0 = Int('sensor_current_bpm_0')
```

```

sensor_OUT_value_0 = Int('sensor_OUT_value_0')
sensor_OUT_reqRead_0 = Bool('sensor_OUT_reqRead_0')
sensor_OUT_reqWrite_0 = Bool('sensor_OUT_reqWrite_0')
monitor_IN_value_0 = Int('monitor_IN_value_0')
monitor_IN_reqRead_0 = Bool('monitor_IN_reqRead_0')
monitor_IN_reqWrite_0 = Bool('monitor_IN_reqWrite_0')
_SafetyController_0_buffered_status_0 = Int('_SafetyController_0_buffered_status_0')
_SafetyController_0_has_data_0 = Int('_SafetyController_0_has_data_0')

```

Property under Verification The property is `monitor_IN_value==0`. Similarly, we list the excerpt of the verification output.

Listing 1.4. Excerpt of the output (MedicalSystem.z3-check.py)

```

-----
Step 59:
    sensor_current_bpm:110
    sensor_OUT_value:105
    sensor_OUT_reqRead:False
    sensor_OUT_reqWrite:False
    monitor_IN_value:0
    monitor_IN_reqRead:True
    monitor_IN_reqWrite:True
    _SafetyController_0_buffered_status:1
    _SafetyController_0_has_data:1
-----
Step 60:
    sensor_current_bpm:110
    sensor_OUT_value:105
    sensor_OUT_reqRead:False
    sensor_OUT_reqWrite:False
    monitor_IN_value:1
    monitor_IN_reqRead:False
    monitor_IN_reqWrite:False
    _SafetyController_0_buffered_status:1
    _SafetyController_0_has_data:0
-----

```

6 Results, Limitations and Future Work

6.1 Evaluation and Discussion

We evaluate the translation along three dimensions: *expressiveness*, *semantic preservation* and *performance*.

Expressiveness The translator supports a comprehensive subset of Mediator:

- **Data Types:** Primitives (Int, Real, Bool), Bounded Integers, and arbitrarily nested complex types (Structs, Arrays) via recursive flattening.

- **Control Flow:** Guards, assignments, and function calls (inlined).
- **Properties:** A subset of Linear Temporal Logic (LTL) including Globally (G), Finally (F), Next (X), and Until (U), allowing for the verification of complex temporal behaviors beyond simple invariants.

Semantic Preservation For the targeted subset, the encoding is sound with respect to Bounded Model Checking. Every counterexample found by Z3 within bound k corresponds to a valid execution trace in the concrete Mediator model. Additionally, the tool optionally verifies type safety constraints (e.g., integer overflows) to ensure runtime robustness.

Performance Optimization Several strategies are employed to mitigate the complexity of SMT solving:

- **Efficient Frame Axioms:** We explicitly generate frame conditions ($x_{t+1} = x_t$) only for variables not modified in a transition, reducing constraint density.
- **Flattening vs. ADTs:** By flattening complex types into primitive scalars, we avoid the overhead associated with Z3’s Algebraic Data Types (ADTs).
- **Bounded Unrolling:** The SSA-like encoding facilitates constraint propagation by modern SMT solvers.

6.2 Limitations

- **Boundedness:** As a BMC tool, it cannot prove properties for infinite horizons without extensions like k-induction. It is complete for finding bugs up to depth k but incomplete for proving correctness.
- **Feedback Granularity:** While the tool detects violations, mapping the raw SMT counterexample back to the precise source code location or visual trace is currently manual and requires domain expertise.
- **Dynamic Constructs:** Unbounded queues and dynamic process creation are not supported due to the static nature of the flattening approach.

6.3 Future Works

- **Boundedness:** As a BMC tool, it cannot prove properties for infinite horizons without extensions like k-induction. It is complete for finding bugs up to depth k but incomplete for proving correctness.
- **Feedback Granularity:** While the tool detects violations, mapping the raw SMT counterexample back to the precise source code location or visual trace is currently manual and requires domain expertise.
- **Dynamic Constructs:** Unbounded queues and dynamic process creation are not supported due to the static nature of the flattening approach.

7 Conclusion

We presented a translation tool from a practical subset of the *Mediator* language to Z3 Python encodings, enabling SMT-based verification of *Mediator* models. The tool supports state/time-indexed encodings for bounded model checking and was applied to two representative models to demonstrate feasibility. While bounded checks cannot prove unbounded properties in general, this approach provides a practical path for early bug finding and model assurance. Future work aims to broaden language coverage, improve scalability and add inductive proving capabilities.

Notes and Comments. This whole project is homework from Programming Techniques and Methodology taught by Professor Meng Sun from School of Mathematical Sciences, Peking University. We would like to express gratitude to Yi Li, Weidi Sun and Meng Sun for proposing the language *Mediator* so that components and systems can be modeled separately and precisely, and also to Professor Sun, for getting our focus on this topic.

The most essential documents of the project should be Z3Generator.java, a script which translate teh *Mediator* to Z3Python encoding; generated testbench: testbench_2.py, an unrolled transition system with initial conditions, transition disjunctions and a check of the safety property.

More details about the development of the project can be accessed in the DEVELOPMENT.md https://github.com/tanzc628/mediator_z3pytranslation/blob/main/DEVELOPMENT.md, including introduction & background information, project structure, implementation status and future work.

References

1. Our code can be accessed on https://github.com/tanzc628/mediator_z3pytranslation
2. Yi Li, Weidi Sun, Meng Sun: *Mediator*: A component-based modeling language for concurrent and distributed systems. Science of Computer Programming 192 (2020) 102438. <https://doi.org/10.1016/j.scico.2020.102438>
3. Nikolaj Bjørner, Leonardo de Moura, Lev Nachmanson, and Christoph Wintersteiger: Programming Z3, Microsoft Research <https://z3prover.github.io/papers/programmingz3.html>
4. Nikolaj Bjørner, Clemens Eisenhofer, Arie Gurfinkel, Nuno P. Lopes, Leonardo de Moura, Lev Nachmanson, Christoph Wintersteiger <https://z3prover.github.io/papers/z3internals.html>
5. Yi Li and Meng Sun: Generating Arduino C Codes from *Mediator* in: It's All About Coordination: Essays to Celebrate the Lifelong Scientific Achievements of Farhad Arbab, in: LNCS, vol. 10865, Springer, 2018, pp. 174-188. https://doi.org/10.1007/978-3-319-90089-6_12
6. A list of *Mediator* <https://github.com/liyi-david/Mediator-Proposal>
7. de Moura, L., Bjørner, N. (2008). Z3: An Efficient SMT Solver. In: Ramakrishnan, C.R., Rehof, J. (eds) Tools and Algorithms for the Construction and Analysis of Systems. TACAS 2008. Lecture Notes in Computer Science, vol 4963. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-540-78800-3_24

8. Biere, A., Cimatti, A., Clarke, E., Zhu, Y. (1999). Symbolic Model Checking without BDDs. In: Cleaveland, W.R. (eds) Tools and Algorithms for the Construction and Analysis of Systems. TACAS 1999. Lecture Notes in Computer Science, vol 1579. Springer, Berlin, Heidelberg. https://doi.org/10.1007/3-540-49059-0_14
9. The Yosys Project <https://github.com/YosysHQ/yosys>
10. DeLine R., Leino K.R.M., “BoogiePL: A typed procedural language for checking object-oriented programs”, MSR Technical Report, 2005.
<https://www.microsoft.com/en-us/research/publication/boogiepl-a-typed-procedural-language-for-checking-object-oriented-programs/>
11. "IEEE Draft Standard for Universal Verification Methodology Language Reference Manual," in IEEE Std P1800.2/D7, November 2016 , vol., no., pp.1-504, 1 Jan. 2016. <https://ieeexplore.ieee.org/servlet/opac?punumber=7755719>
12. "IEEE Standard for Property Specification Language (PSL) - Redline," in IEEE Std 1850-2010 (Revision of IEEE Std1850-2005) - Redline , vol., no., pp.1-188, 6 April 2010 <https://doi.org/10.1109/IEEESTD.2010.5948310>.
13. "IEEE Standard for Property Specification Language (PSL) - Redline," in IEEE Std 1850-2010 (Revision of IEEE Std 1850-2005) - Redline , vol., no., pp.1-188, 6 April 2010. <https://ieeexplore.ieee.org/servlet/opac?punumber=9404013>
14. Guijarro, L. (2004). Analysis of the Interoperability Frameworks in e-Government Initiatives. In: Traunmüller, R. (eds) Electronic Government. EGOV 2004. Lecture Notes in Computer Science, vol 3183. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-540-30078-6_7
15. Clarke, E., Kroening, D., Lerda, F. (2004). A Tool for Checking ANSI-C Programs. In: Jensen, K., Podelski, A. (eds) Tools and Algorithms for the Construction and Analysis of Systems. TACAS 2004. Lecture Notes in Computer Science, vol 2988. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-540-24730-2_15
16. Barnett, M., Chang, B.Y.E., DeLine, R., Jacobs, B., Leino, K.R.M. (2006). Boogie: A Modular Reusable Verifier for Object-Oriented Programs. In: de Boer, F.S., Bonsangue, M.M., Graf, S., de Roever, W.P. (eds) Formal Methods for Components and Objects. FMCO 2005. Lecture Notes in Computer Science, vol 4111. Springer, Berlin, Heidelberg. https://doi.org/10.1007/11804192_17
17. David Harel, Statecharts: a visual formalism for complex systems, Science of Computer Programming, Volume 8, Issue 3, 1987, Pages 231-274, ISSN 0167-6423, [https://doi.org/10.1016/0167-6423\(87\)90035-9](https://doi.org/10.1016/0167-6423(87)90035-9)
18. Edmund M. Clarke, Thomas A. Henzinger, Helmut Veith, Roderick Bloem:Handbook of Model Checking <https://doi.org/10.1007/978-3-319-10575-8>
19. Bradley, A.R. (2011). SAT-Based Model Checking without Unrolling. In: Jhala, R., Schmidt, D. (eds) Verification, Model Checking, and Abstract Interpretation. VMCAI 2011. Lecture Notes in Computer Science, vol 6538. Springer, Berlin, Heidelberg https://doi.org/10.1007/978-3-642-18275-4_7.